

✓ MICA Sanity Check — Colab

Purpose: confirm MICA's actual DL inference pipeline runs end-to-end (preprocessing → neural network → clustering/backbone tracing → PULCHRA full-atom build) without crashing. **Not** a benchmark run — no ground-truth comparison, no metrics.

Uses MICA's own official Quick Start sample data (EMD-15635 / PDB 8AT6). Note: this entry is literally in MICA's own training set (confirmed against Supplementary Table S6), so a good-looking result here means nothing about generalization — this is pure plumbing verification, not a real test case. Real scoring happens later, on our own leakage-screened 12-entry benchmark set, not this demo.

Before running: Runtime → Change runtime type → Hardware accelerator: **GPU (T4)**. If a "High-RAM" / shape option is available, select it — the local (Mac, 16GB RAM) attempt at this exact same sanity check OOM-killed twice on this entry's 512 grid samples (~13GB+ estimated footprint), which is the whole reason we moved this to Colab.

Disconnection risk: the full run (preprocessing + DL inference at batch_size=1 + post-processing) takes ~25-45+ minutes. Free-tier Colab disconnects idle/backgrounded sessions, and `/content` is wiped on a full disconnect — losing everything (cloned repo, installed packages, downloaded model/data) and forcing a full restart from cell 1. **Keep the browser tab open and in the foreground for the whole run** rather than switching away or letting the laptop sleep. If it does disconnect, check with `!ls /content/MICA/trained_models/ 2>/dev/null` first — if that's empty, it's a fresh session and needs `Runtime → Run all` from the top; if it still shows the model file, the same backend session survived and you only need to re-run the inference cell.

```
# Confirm GPU + RAM before doing anything expensive
import torch, subprocess
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print("VRAM (GB):", round(torch.cuda.get_device_properties(0).total_memory / 1024 / 1024, 1))
print()
print(subprocess.run(["cat", "/proc/meminfo"], capture_output=True,
```

```
CUDA available: True
GPU: Tesla T4
VRAM (GB): 15.6
```

```
MemTotal:      13286936 kB
```

```
# Clone MICA
!git clone https://github.com/jianlin-cheng/MICA.git
%cd MICA
```

```
Cloning into 'MICA'...
remote: Enumerating objects: 277, done.
remote: Counting objects: 100% (137/137), done.
remote: Compressing objects: 100% (109/109), done.
remote: Total 277 (delta 78), reused 64 (delta 26), pack-reused 140
Receiving objects: 100% (277/277), 190.99 MiB | 16.11 MiB/s, done.
Resolving deltas: 100% (93/93), done.
Updating files: 100% (72/72), done.
/content/MICA
```

```
# Python deps. cython installed FIRST and separately – avoids the e
# failure we hit locally (freesasa, pulled in transitively by atom3
# build via setuptools' legacy on-the-fly cython auto-fetch; pre-in
# cython directly sidesteps that broken code path entirely).
#
# superpose3d PINNED to 1.1.1 (matching environment.yml) rather tha
# modeler.py:202 calls Superpose3D(...)[0][0], assuming a nested re
# that breaks against the current PyPI release (1.5.0) with
# "IndexError: invalid index to scalar variable". Found this the ha
# after a full successful DL inference run, so don't remove this pi
!pip install -q cython
!pip install -q biopython mrcfile atom3d "superpose3d==1.1.1" sciki
```

```

270.1/270.1 kB 9.7 MB/s
Preparing metadata (setup.py) ... done
3.2/3.2 MB 82.6 MB/s eta
45.0/45.0 kB 4.5 MB/s et
52.7/52.7 kB 3.9 MB/s et
400.8/400.8 MB 4.1 MB/s
8.9/8.9 MB 103.3 MB/s et
140.1/140.1 kB 14.0 MB/s
346.8/346.8 kB 32.5 MB/s
46.8/46.8 kB 4.3 MB/s et
2.2/2.2 MB 96.1 MB/s eta
82.2/82.2 kB 8.7 MB/s et
4.9/4.9 MB 126.1 MB/s et
56.8/56.8 kB 6.6 MB/s et
Building wheel for freesasa (setup.py) ... done
```

```
# Trained model weights (154.7MB, from the paper's Zenodo record)
!curl -sL -o trained_models.tar.gz "https://zenodo.org/records/1575
!tar -xzf trained_models.tar.gz
!rm trained_models.tar.gz
!ls trained_models/
```

MICA_best_model.pth

```
# Official Quick Start sample input (21.1MB compressed on Zenodo --
# is NOT the same file as the 272MB `input.tar` sitting locally; th
# already been decompressed, this one starts fresh and unpacks to t
# content). Includes EMD-15635 map, 8AT6 FASTA, and pre-docked AF3
# (the paper's authors bundled fully-docked demo data, so this test
# map+AF3 input fusion, not the degraded map-only fallback).
!curl -sL -o input.tar.gz "https://zenodo.org/records/15756654/file
!tar -xzf input.tar.gz
!rm input.tar.gz
!ls input/15635/
```

15635_af3_docked.pdb 8at6.fasta AF3_structures emd_15635.map

```
# PULCHRA -- unlike the Mac (which needed a custom arm64 compile si
# bundled binaries are Linux x86-64 / 32-bit macOS, neither usable
# Colab IS Linux x86-64, so the repo's own bundled binary should ju
!chmod +x modules/pulchra304/src/pulchra
!modules/pulchra304/src/pulchra 2>&1 | head -5
```

PULCHRA Protein Chain Restoration Algorithm version 3.04
Usage: modules/pulchra304/src/pulchra [options] <pdb_file>
The program default input is a PDB file.
Output file <pdb_file.rebuild.pdb> will be created as a result.
Valid options are:

```
# Force batch_size=1 -- utils/predict.py's own _calculate_optimal_b
# badly underestimates real GPU memory needs (predicted batch=8 fit
# budget on a 14.6GB T4; actual peak usage for ONE such batch was 1
# causing an immediate CUDA OOM on the very first batch). No CLI fl
# for this, so patching the source directly is the only way. Confir
# at batch_size=1 (512/512 samples, ~21 min, no OOM).
!sed -i 's/self\.optimal_batch_size = self\._calculate_optimal_batc
!grep -n "optimal_batch_size = " utils/predict.py
```

```
81:         self.optimal_batch_size = 1
201:         self.optimal_batch_size = 1
```

```
# The actual sanity-check run -- README's Quick Start command, --de
# since we have a real GPU here (this is the whole point of moving
# local Mac, which has no CUDA at all).
```

```
!python run.py \
  -m input/15635/emd_15635.map \
  -f input/15635/8at6.fasta \
  -a input/15635/AF3_structures \
  --run_pulchra --pulchra_path=modules/pulchra304/src/pulchra \
  --resolution=3.7 --device cuda \
  -o output
```

```
2026-09-27 19:54:01 - INFO - Processing sequence 8AT6_3 with 273 r
2026-09-27 19:54:16 - INFO - AF3 matching results: 251 matches, to
2026-09-27 19:54:16 - INFO - Processing sequence 8AT6_2 with 309 r
2026-09-27 19:54:39 - INFO - AF3 matching results: 295 matches, to
2026-09-27 19:54:39 - INFO - ✓ AF3 structure-based alignment compl
=====
```

BUILDING INITIAL MODEL

```
=====
2026-09-27 19:54:39 - INFO - Starting initial modeling...
2026-09-27 19:54:39 - INFO - Starting initial model building with :
2026-09-27 19:54:39 - INFO - Processing 456 traces for sequence 8A
2026-09-27 19:54:39 - INFO - Chain list: ['A', 'D']
2026-09-27 19:54:39 - INFO - Processing 309 traces for sequence 8A
2026-09-27 19:54:39 - INFO - Chain list: ['B', 'E']
2026-09-27 19:54:39 - INFO - Processing 273 traces for sequence 8A
2026-09-27 19:54:39 - INFO - Chain list: ['C', 'F']
2026-09-27 19:54:39 - INFO - Saving initial model: output/15635_8a
2026-09-27 19:54:39 - INFO - ✓ Initial model building completed in
```

RUNNING GAP FILLING

```
2026-09-27 19:54:39 - INFO - Starting gap filling...
2026-09-27 19:54:39 - INFO - Starting gap filling process...
2026-09-27 19:54:40 - INFO - Processing gap 1/6: positions 413->450
2026-09-27 19:54:41 - INFO - Processing gap 2/6: positions 413->450
2026-09-27 19:54:41 - INFO - Processing gap 3/6: positions 164->230
2026-09-27 19:54:45 - INFO - Processing gap 4/6: positions 164->240
2026-09-27 19:54:49 - INFO - Processing gap 5/6: positions -1->77
2026-09-27 19:54:51 - INFO - Processing gap 6/6: positions -1->77
2026-09-27 19:54:55 - INFO - Processing gap 1/6: positions -1->11
2026-09-27 19:54:56 - INFO - Processing gap 2/6: positions -1->11
2026-09-27 19:54:56 - INFO - Processing gap 3/6: positions 139->150
2026-09-27 19:54:57 - INFO - Processing gap 4/6: positions 137->150
2026-09-27 19:54:59 - INFO - Processing gap 5/6: positions 230->300
2026-09-27 19:54:59 - INFO - Processing gap 6/6: positions 228->300
2026-09-27 19:55:02 - INFO - Processing gap 1/7: positions 266->270
2026-09-27 19:55:02 - INFO - Processing gap 2/7: positions 266->270
2026-09-27 19:55:02 - INFO - Processing gap 3/7: positions 133->140
2026-09-27 19:55:03 - INFO - Processing gap 4/7: positions -1->8
2026-09-27 19:55:03 - INFO - Processing gap 5/7: positions -1->12
2026-09-27 19:55:04 - INFO - Processing gap 6/7: positions 192->200
2026-09-27 19:55:04 - INFO - Processing gap 7/7: positions 191->200
2026-09-27 19:55:06 - INFO - Saving gap filled model: output/15635
2026-09-27 19:55:06 - INFO - ✓ Gap filling completed in 26.95 seconds
```

RUNNING PULCHRA

```
2026-09-27 19:55:06 - INFO - Starting Pulchra reconstruction...
2026-09-27 19:55:06 - INFO - Starting Pulchra processing for PDB file
2026-09-27 19:55:06 - INFO - Splitting PDB file by chains...
2026-09-27 19:55:06 - INFO - Running Pulchra reconstruction on chains
2026-09-27 19:55:06 - INFO - Combining rebuilt chains into all-atom
2026-09-27 19:55:06 - INFO - ✓ Pulchra processing completed. All-atom
2026-09-27 19:55:06 - INFO - ✓ Pulchra completed in 0.44 seconds
2026-09-27 19:55:06 - INFO - Recording timing information: output/
```

MICA PIPELINE COMPLETED SUCCESSFULLY

 Total processing time: 2670.67 seconds

```
# Confirm something was actually produced
!find output -type f | head -20
```

```
from google.colab import files
```

```
files.download('output/15635_8at6_MICA_all_atom_model.pdb')
```

```
-----  
-----  
FileNotFoundError                                Traceback (most recent  
call last)  
/tmp/ipykernel_1926/3185622351.py in <cell line: 0>()  
----> 1 files.download('output/15635_8at6_MICA_all_atom_model.pdb')  
  
/usr/local/lib/python3.13/dist-packages/google/colab/files.py in  
download(filename)  
    228     if not _os.path.exists(filename):  
    229         msg = 'Cannot find file: {}'.format(filename)  
--> 230         raise FileNotFoundError(msg) # pylint:  
disable=undefined-variable  
    231  
    232     comm_manager = IPython.get_ipython().kernel.comm_manager
```

Start coding or [generate](#) with AI.